RETREIVE Application Documentation

1. Karaoke Technology Stack

For the Voice Karaoke Reader, we utilize the native Web Speech API (`window.SpeechRecognition` / `window.webkitSpeechRecognition`).

- Real-time Recognition: The speech engine runs locally in the browser with continuous = true and interimResults = true.
- Dynamic Alignment: As speech chunks are received, they are processed and compared to passage words.
- Cursor Highlighting: Words matching the user's spoken stream are highlighted dynamically in green, and the reading cursor automatically increments to advance paragraph highlights.

---

2. Key Modules & Implementations

Component	What it Accomplishes	How it's Done
PDF Parser	Extracts text structure, tokenizes words, and prepares paragraphs.	Client-side parser with word-indexing metadata so each token is uniquely trackable.
Karaoke Engine	Controls reader tracking, auto-scrolls text blocks, and highlights read items.	Reactive cursors matched with continuous interim transcript buffers from browser recognition.
Progress & XP System	Persists streaks, awards levels/badges, and stores	Batched writes in `src/lib/db.ts` update user stats, leaderboard rankings, and unlocked badge arrays

Component	What it Accomplishes	How it's Done
	session records.	concurrently.
Paywall Gateway	Restricts free accounts to 1 trial session before asking for upgrade.	Checks `totalSessions` in Firestore and `has_completed_session` in localStorage to restrict secondary uploads until subscription is complete.
Mascot Interaction	Enhances engagement with animation and tips.	Loopable inline video player that responds dynamically to mouse hover (lift-on-hover card styling).

---

3. Future Roadmap

A. Integration of Deepgram API for Voice Transcription

- **Why: The native Web Speech API is browser-dependent (mostly Chrome/Edge) and has performance variations based on local microphone noise.**
- **How: Implement a server-side or SDK-based Deepgram Real-time Transcription pipeline. Deepgram offers extremely low latency, superior accent model support, and precise word-level timestamps.**

B. Passage Audio Playback (Listen Mode)

- **Why: To accommodate audio-visual learners who want to hear complex medical terminology pronounced correctly.**
- **How: Integrate Text-to-Speech (TTS) engines (e.g., ElevenLabs or OpenAI TTS) to generate high-fidelity audio read-alongs for selected text.**

C. Granular Analytics Dashboards

- **Why: Students need insights on which specific medical topics (e.g., Biochemistry vs. Psychosociology) they struggle to articulate.**

- **How: Log reading speed variances and correct answers by tagging uploaded PDFs with topics, storing analytical metrics per category in Firestore, and plotting them with Recharts.**

D. Offline Reading Synchronization

- **Why: Students study on subways/airplanes without internet access.**
- **How: Extend the offline storage queue to store cached PDF text in browser IndexedDB, allowing offline read sessions to queue up XP updates, which sync with Firestore once the internet connection returns.**